

Proyecto Spark

Paralelismo y Sistemas Distribuidos

Grado en Ciencia e Ingeniería de Datos

Universidad Politécnica de Cataluña

Trabajo realizado por

David González Acén

Ángel Morales Cirera

David Torrecilla Tolosa

Mayo de 2025

Tabla de contenido

1.	Introducción.....	3
2.	Descripción del experimento.....	4
	Descripción general	4
	Detalles y pasos a seguir.....	5
	Material entregado.....	5
	Consideraciones.....	6
3.	Resultados.....	7
4.	Conclusiones.....	9
5.	Bibliografía	10

1. Introducción

En este trabajo se estudia el rendimiento de Spark en función del tamaño de los datos de entrada. Se comparan tres versiones de la misma aplicación: secuencial con Python, distribuida con coreSpark y distribuida con sparkSQL, ejecutadas en cuatro instancias de máquinas virtuales de Amazon Web Services EC2.

El objetivo principal es ver cómo se comporta cada versión conforme aumentamos el tamaño, y comparar las diferentes versiones para ver cuáles son los umbrales aproximados a partir de los cuales nos interesa más usar una versión u otra. La métrica de referencia es el tiempo, que normalmente es proporcional a la energía consumida, ya que se usa la misma configuración de *hardware* en todos los casos (cuatro instancias de EC2).

Es de esperar que para pocos datos sea más conveniente usar el programa secuencial, así que esto es una afirmación que se trata de verificar (de forma experimental). Por otro lado, no tenemos hipótesis a priori de cómo se compararán la versión de coreSpark y sparkSQL, como mucho podemos esperar que sean más parecidas entre ellas de lo que lo son con la secuencial.

2. Descripción del experimento

Descripción general

Se han ejecutado dos veces un total de quince combinaciones de la versión del programa y del tamaño de los datos de entrada: cada una de las tres versiones con una entrada de tamaño x1, x2, x4, x8 y x16 en relación al archivo infoActors.csv. En número de líneas, son unas 2,3; 4,7; 9,4; 18,7 y 37 millones de líneas respectivamente, y pesan unos 150, 300, 600, 1200 y 2400 MB respectivamente. Cada uno de los archivos de input se ha conseguido copiando múltiples veces el archivo original (eliminando las cabeceras de en medio).

Estos datos de entrada se han proporcionado por la profesora Yolanda Becerra en un enlaceⁱ. Consiste en un archivo de extensión .csv que contiene datos de IMDBⁱⁱ, sobre un conjunto de películas. Cada fila consiste en una pareja actor – director que trabajó en una película. Tenemos, para cada fila, los IDs de la película, del actor y del director, sus nombres, y la puntuación media (también el número de votos, pero no se ha utilizado).

El objetivo de los programas implementados es dar la lista (o tabla) de las veinte parejas actor – director que más se repiten, junto con el número de repeticiones y las puntuaciones media, mínima y máxima de entre todas sus películas.

Este comportamiento se ha conseguido con tres versiones. La primera es puro Python (Python_actorDirector.py), y se basa en aplicar los “mapeados” iterativamente. La segunda es usando Spark (RDD_actorDirector.py), pero limitado al formato RDD del paquete pyspark. Consiste en hacer las transformaciones necesarias y acabar con una acción. La tercera versión es usando dataframes de pyspark.sql y se forma, esencialmente, de una sentencia SQL.

Detalles y pasos a seguir

La ejecución se ha utilizado usando Wall para medir el tiempo, en cuatro máquinas virtuales Ubuntu de AWS EC2. Las instancias son m5.large, cada una con 2 CPUS virtuales, 8 GB de RAM y 30 de almacenamiento. Una de ellas se ha utilizado como *driver* (master), y el resto como *executors* (slaves).

Las cuatro requieren una configuración inicial: copiar los cinco archivos de input, configurarlas de forma que se puedan interconectar entre ellas (usando la misma pareja de claves pública y privada, y desde cualquier puerto), e instalar Spark, que se ha hecho usando el script `automatic_deploy.sh`, proporcionado por la profesora, desde el master.

El master requiere también que se copien los programas de Python, que se añadan las 4 IPs públicas de las instancias en `spark-3.5.5-bin-hadoop3/conf/workers`, y que se ejecute el script `spark-3.5.5-bin-hadoop3/sbin/start-all.sh`. Posteriormente, buscar el log del master y apuntar la url del master.

Hecha la configuración, se ha usado el script `run_tests.sh`, que ejecuta cada versión con cada input, guardando todo el output en un archivo en la carpeta logs del master. Estas salidas contienen el *elapsed time* de cada ejecución, que se han puesto en el código del programa `graficador.py`, encargado de hacer el gráfico.

Como ya hemos mencionado, se han hecho dos ejecuciones de cada combinación, es decir ejecutar el script dos veces. El gráfico resultante es la media de los tiempos de ambas ejecuciones. Hay tres líneas, una para cada versión de programa. La variable independiente es el tamaño de la entrada.

Material entregado

Junto con este PDF, se ha entregado:

- Los tres programas de Python.
- Los scripts `run_tests.sh`, `automatic_deploy.sh`.
- `graficador.py`, que sirve para hacer los gráficos de los tiempos.
- `/logs`, con las salidas.

Consideraciones

La salida de los programas debe ser la misma, salvo cambios en el formato y el orden de las parejas con el mismo número de repeticiones. Asimismo, el tamaño de la entrada es el factor multiplicador del número de repeticiones (pero el resto de datos de la salida deben ser los mismos).

El rendimiento de Spark no debe estar limitado por una mala configuración (poca memoria RAM, pocas particiones, transformaciones innecesarias...). Se debe comprobar el número de particiones y ajustarlo, teniendo en cuenta que se recomiendan entre 2 y 4 particiones por nodo, y que pesen entre 128 y 400 MB. En nuestro caso, nos enfocaremos en el primer criterio, porque los archivos pesan muy poco (por ejemplo, el caso X1 cabe en dos particiones).

Por la construcción de la sentencia SQL, es importante usar el `df.repartition(N, 'columna')` con una de las columnas que se usa para el GROUP BY.

3. Resultados

Tras ejecutar, comprobamos las salidas. Vemos coherencia en los datos (mismos datos, factor del tamaño de entrada correcto, son los datos que esperábamos como resultado de los programas).

Comprobamos también que Spark crea correctamente tantos ejecutores como cores virtuales disponibles, y coge memoria de sobras. Por ejemplo:

```
Executor added: app-... on worker-... with 2 core(s)
```

```
Granted executor ID app-... on hostPort ... with 2 core(s), 1024.0 MiB RAM
```

En nuestro caso esperamos ocho mensajes así, en cada output de RDD y DF.

También comprobamos el número de particiones. Para pocos datos de entrada no tendrá tanto efecto, pero sí es importante para las entradas grandes. Por ejemplo, para la versión más grande (X16), vemos que la versión con dataframes utiliza entre 19 y 25 particiones (Starting task 18.0 ...; Starting task 24.0) en las operaciones más pesadas.

La versión de RDD usa 74 particiones. Hemos probado de usar un repartition(32) al principio, pero para la versión X16, el tiempo se incrementa a los dos minutos, con lo cual hemos dejado que Spark decida el número de particiones.

Hacemos la segunda ejecución, y copiamos los tiempos en el graficador.py. Ejecutamos el programa, y tenemos el gráfico de la media de los tiempos de cada versión según el tamaño de la entrada.

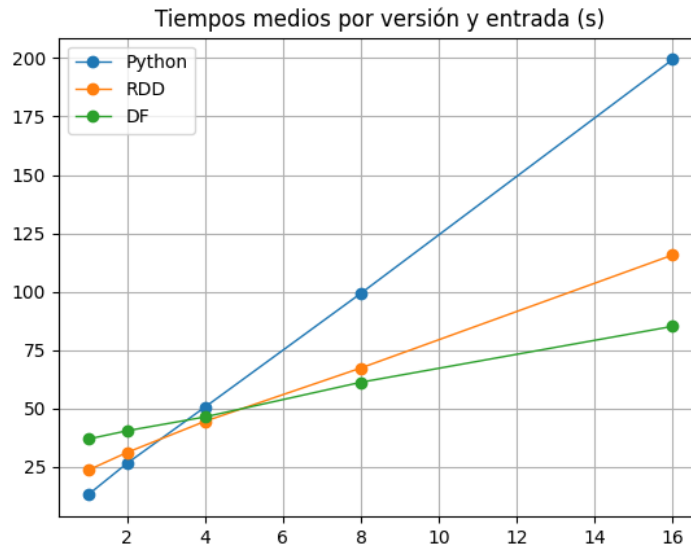


Ilustración 1 - Tiempos medios, entre dos ejecuciones, según el tamaño de la entrada para cada versión

A partir del gráfico, vemos como el tiempo crece lineal o casi linealmente en ambas versiones, pero cada una con su propia pendiente. La versión de Python es la que más sufre un incremento en el tiempo según aumentamos el tamaño de la entrada, con diferencia respecto a las otras dos. La versión con RDD es la que se ve ligeramente más afectada que la versión con dataframes. Para ponerlo en perspectiva, con la entrada X4, los tres tardan unos 50 segundos, sin embargo, al cuadruplicar la entrada (X16), el tiempo de Python también se cuadruplica, mientras que el de RDD queda en un poco más del doble, y la de dataframes un poco menos del doble.

Por otro lado, si nos fijamos en las entradas pequeñas, el orden de los programas se invierte, es decir, Python básico es el más rápido, seguido del de RDD y luego el de dataframes. No es hasta aproximadamente la entrada X4 que vemos el orden invertido.

4. Conclusiones

Podemos concluir principalmente tres afirmaciones:

1. El tiempo incrementa linealmente en cualquiera de las versiones, así que haciendo un test con pocos datos, se puede predecir lo que tardaremos con una entrada proporcionalmente más grande. Lo que cambia entre cada versión es la pendiente de la recta.
2. Si la cantidad de datos es pequeña, la gestión debería hacerse secuencialmente con Python. La definición de “pequeña” depende de diversos factores, principalmente de cuántos nodos usemos, ya que usar más implica más overhead. Usando cuatro nodos, 8 cores, una buena referencia es un archivo que ronde un máximo de 5 millones de líneas o de unos 500 MB.
3. Si la entrada de datos es considerablemente mayor, es conveniente usar `pyspark.sql` para tratar los datos usando dataframes, y según convenga, sentencias SQL. Este módulo es, por lo general, más práctico (aparte de rápido). Por supuesto esto depende de la necesidad que tengamos, ya que los RDDs están en un nivel más bajo y es posible que permitan hacer algunas cosas que los dataframes no.

A parte, podemos tener en cuenta que puede haber un intervalo muy corto en el que sea más rápido usar RDDs que las otras versiones. Puede ser útil si trabajamos siempre con un tamaño fijo de entrada (que en nuestro caso rondaría los 500 MB o 5 millones de líneas).

5. Bibliografía

Diapositivas de los temas de Cloud computing, Hadoop y Spark, de la asignatura de Paralelismo y sistemas distribuidos del Grado en ciencia e ingeniería de datos. No disponible públicamente.

Documentación en línea de Apache Spark,

<https://spark.apache.org/docs/latest/api/python/index.html>

ⁱ https://drive.google.com/file/d/1lu4DU78NRHxIzf6rxupGSdr6rPrJKAw/view?usp=share_link

ⁱⁱ <https://developer.imdb.com/non-commercial-datasets>